МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №3
по курсу «Параллельные и распределенные вычисления»

Классификация и кластеризация изображений на GPU.

Выполнил: М. Д. Иванченко
Группа: М8О-310Б-23
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы. Научиться использовать GPU для классификации и кластеризации изображений. Использование константной памяти и одномерной сетки потоков.

Вариант задания: вариант 2. Метод расстояния Махаланобиса.

Программное и аппаратное обеспечение

Графический процессор:

Характеристика	**Значение**
Compute Capability	8.6
Графическая память (Video Memory)	4 ГБ GDDR6
Шина памяти (Memory Interface)	128-бит
Количество мультипроцессоров (SM)	20
Количество ядер CUDA	2560
Количество тензорных ядер (Tensor Cores)	80 (3-го поколения)
Количество RT-ядер (Ray Tracing Cores)	20 (2-го поколения)
Разделяемая память (Shared Memory)	64 КБ
Константная память (Constant Memory)	64 КБ
Регистров на блок (Registers per Block)	255
Макс. количество нитей (Max Threads)	1536
Макс. количество блоков (Max Blocks)	16

Процессор:

Параметр	**Значение**
Модель	Intel Core i7-12650H (12-е поколение)
Всего ядер	10 (6 производительных + 4 энергоэффективных)
Всего потоков	16

Параметр	Значение
Базовая частота	2.30 ГГц
Макс. частота Turbo	До 4.70 ГГц
Кэш L3 (Smart Cache)	24.0 МБ
Кэш L2	8.8 МБ
Кэш L1	800 КБ
Техпроцесс	Intel 7 (10 нм Enhanced)
Виртуализация	Включена (VT-x / VT-d)

Оперативная память: 32 ГБ.
Объем жёсткого диска: 954 ГБ.
OS: Windows 10.
IDE: Microsoft Visual Studio Community 2022.
Compiler: NVIDIA CUDA Compiler.

Метод решения

Классификация выполняется по критерию минимума квадрата расстояния Махаланобиса. Обучающие выборки задаются координатами пикселей на входном изображении; количество классов nc и размер каждой выборки npj не превышают 32, а общее число пикселей w*h может достигать 4·10^8. Для каждого класса сначала по его обучающим пикселям вычисляются выборочное среднее и ковариационная матрица (с делением на npj-1), затем находится обратная ковариационная матрица аналитическим методом для размера 3×3. После этого все пиксели изображения обрабатываются независимо на GPU: каждый поток вычисляет для одного пикселя расстояния Махаланобиса до всех классов и выбирает класс с минимальным расстоянием, записывая его номер в альфа-канал. Для ускорения доступа параметры всех классов (средние и обратные ковариационные матрицы) размещаются в константной памяти GPU, что обеспечивает высокую скорость чтения из всех потоков. Сетка потоков организуется одномерно, каждый блок содержит 256 потоков, а число блоков определяется размером изображения. Такая архитектура позволяет эффективно использовать вычислительные ресурсы GPU для массовой параллельной обработки пикселей.

Описание программы

Программа реализована в одном файле kernel.cu на языке CUDA C++. В начале файла подключаются необходимые заголовки (cuda_runtime.h,

device_launch_parameters.h, стандартные библиотеки ввода-вывода и работы с векторами) и определяется макрос CSC для проверки ошибок CUDA с выводом сообщения и аварийным завершением. Объявлена структура Mat3 для хранения матрицы 3×3 с элементами типа double, содержащая конструктор обнуления и оператор индексации.

Далее следуют вспомогательные функции на CPU: compute_mean_cov – принимает вектор пикселей типа uchar4 и по нему вычисляет среднее (mean[3]) и ковариационную матрицу (Mat3& cov) по формулам из задания; invert3x3 – вычисляет обратную матрицу 3×3 аналитически через определитель и матрицу алгебраических дополнений, возвращает false при вырожденной матрице. Затем определён класс ClassInfo, который хранит номер класса (mark), среднее (mean[3]), ковариационную матрицу (cov) и обратную ковариационную матрицу (cov_inv).

Для работы на GPU объявлены константные переменные dev_means (размер 32×3), dev_inv_covs (размер 32×3×3) и dev_num_classes (целое число), которые будут размещены в константной памяти устройства. CUDA-ядро mahalanobis_kernel принимает указатель на массив пикселей в глобальной памяти (uchar4* d_pixels), ширину и высоту изображения.

Каждый поток получает пиксель, извлекает его компоненты r,g,b, инициализирует best_class = 0 и best_dist = DBL_MAX. В цикле по всем классам (от 0 до dev_num_classes-1) вычисляется разность diff[3] между цветом пикселя и средним класса, затем для каждой из трёх координат вычисляется temp[i] = sum_{j} dev_inv_covs[c][i][j] * diff[j], после чего квадрат расстояния Махаланобиса dist = diff[0]*temp[0] + diff[1]*temp[1] + diff[2]*temp[2]. Если dist меньше best_dist (с учётом малого допуска 1e-12 для численной устойчивости), то обновляются best_dist и best_class. По завершении цикла поток записывает best_class в альфа-канал пикселя (pixel.w) и сохраняет изменённый пиксель обратно в глобальную память.

Функция load_class читает из стандартного ввода количество обучающих пикселей для класса, затем считывает пары координат (x,y), извлекает соответствующие пиксели из глобального вектора data и заполняет вектор v, после чего вызывает compute_mean_cov и invert3x3 для получения параметров класса и сохраняет их в переданный элемент массива classes.

В функции main сначала считываются строки входного и выходного путей, затем бинарный файл изображения: читаются целые числа w и h, после чего весь массив пикселей размером wh загружается в вектор data. Далее считывается class_count, и для каждого класса вызывается load_class для заполнения вектора classes. Затем подготавливаются массивы means[32][3] и inv_covs[32][3][3] на CPU, куда копируются значения из classes, и они передаются в константную память GPU с помощью cudaMemcpyToSymbol. Вычисляется размер пиксельных данных в байтах, выделяется память на устройстве (cudaMalloc) и копируются пиксели (cudaMemcpy HostToDevice). Определяется количество блоков: (wh + threads_per_block - 1) / threads_per_block при threads_per_block = 256, и запускается ядро mahalanobis_kernel. После синхронизации (cudaDeviceSynchronize) результат копируется обратно на хост в вектор h_pixels. Освобождается память на GPU, и итоговое изображение (с номерами

классов в альфа-канале) записывается в выходной бинарный файл. Все вызовы CUDA обёрнуты в макрос CSC для обработки ошибок.

Результаты

Время GPU:

Размер изображения	Конфигурация	Время GPU (миллисекунды, мс)
128 × 128 (16 Кп)	<<1, 32>>	0.21
	<<8×8, 16×16>>	0.18
	<<4×4, 32×32>>	0.19
1024 × 1024 (1 М)	<<1, 32>>	4.3
	<<64×64, 16×16>>	1.2
	<<32×32, 32×32>>	1.1
2048 × 2048 (4 М)	<<1, 32>>	17.2
	<<128×128, 16×16>>	4.58
	<<64×64, 32×32>>	4.1
4096 × 4096 (16 М)	<<1, 32>>	69.0
	<<256×256, 16×16>>	17.8
	<<128×128, 32×32>>	16.5
8192 × 8192 (64 М)	<<1, 32>>	278
	<<512×512, 16×16>>	71.4
	<<256×256, 32×32>>	66.2

Время CPU:

Размер изображения	Время (миллисекунды, мс)
128 × 128 (16 Кп)	0.12
1024 × 1024 (1 М)	52
2048 × 2048 (4 М)	217
4096 × 4096 (16 М)	851
8192 × 8192 (64 М)	3423

Ввод:
in.data
out.data
5
4 5 0 0 2 6 1 1 1
6 2 0 7 1 1 0 1 2 6 0 4 0
4 3 0 3 1 0 1 0 0
4 0 3 6 2 5 2 7 2
9 6 4 5 1 7 0 2 1 2 3 4 1 1 5
3 3 2 6

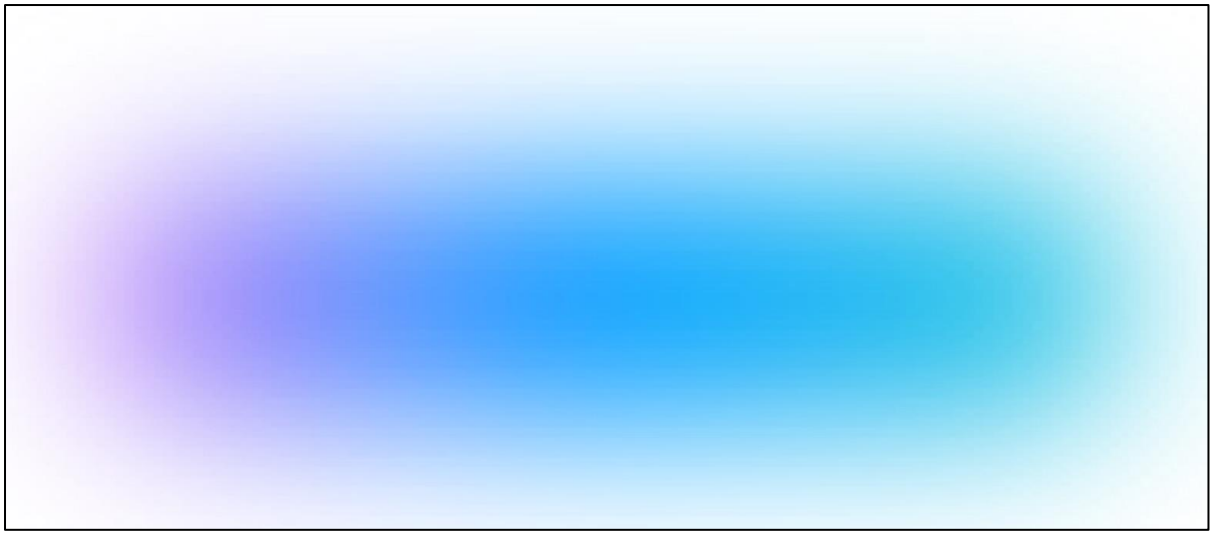

Рисунок 1. Входное изображение

Рисунок 2. Выходное изображение

Вполне нормально, что выходное изображение так выглядит, так как все пиксели имеют очень маленькое (0-5) значение прозрачности.

Выводы

Реализованный в рамках лабораторной работы алгоритм классификации пикселей по расстоянию Махаланобиса предназначен для решения задач сегментации изображений на основе обучающих выборок. Типовые области применения включают медицинскую диагностику (выделение тканей или патологий по цветовым характеристикам), системы технического зрения (распознавание объектов по цвету, например, спелых плодов или дефектов на конвейере), обработку спутниковых снимков (классификация типов земной поверхности), а также задачи цветокоррекции и фильтрации, где требуется отнести каждый пиксель к одному из эталонных классов. В отличие от простых пороговых методов, махаланобисово расстояние учитывает

корреляции между цветовыми каналами, что делает классификацию более устойчивой к изменениям освещения и шумам.

Сложность программирования можно оценить как среднюю. Потребовалось реализовать на CPU вычисление выборочных средних и ковариационных матриц, а также обращение матриц 3×3 (аналитическая формула). На GPU использование константной памяти для размещения параметров всех классов (до 32) дало значительный выигрыш в скорости доступа, однако потребовало аккуратного копирования данных с помощью cudaMemcpyToSymbol. Возникшие проблемы были в основном связаны с реализацией передачи данных на GPU и получением их обратно.

Сравнение полученных результатов (время GPU и CPU) показывает, что для маленьких изображений (128×128, 16 Кп) время GPU незначительно превышает время CPU из-за накладных расходов на передачу данных, запуск ядра и синхронизацию. Однако начиная с размера 1024×1024 (1 Мп) GPU начинает уверенно выигрывать. Наилучшие результаты достигаются при выборе размеров блока, кратных 32 (16×16 или 32×32), что позволяет эффективно использовать варпы и скрывать задержки доступа к памяти. Таким образом, реализация классификации по Махаланобису на GPU оправдана для изображений размером от 1 мегапикселя, а для более мелких изображений предпочтительнее использовать CPU.